

number. The significand of the small number has to shift to the right according to the difference in exponents.

3. *Addition/subtraction*: adds or subtracts the significands of two aligned numbers.
4. *Normalization*: adjusts the result to normalized format. Three types of normalization procedures may be needed:
 - After a subtraction, the result may contain leading zeros in front, as in example 2.
 - After a subtraction, the result may be too small to be normalized and thus needs to be converted to zero, as in example 3.
 - After an addition, the result may generate a carry-out bit, as in example 4.

Our binary floating-point adder design uses a similar algorithm. To simplify the implementation, we ignore the rounding. During alignment and normalization, the lower bits of the significand will be discarded when shifted out. The design is divided into four stages, each corresponding to a step in the foregoing algorithm. The suffixes, 'b', 's', 'a', 'r', and 'n', used in signal names are for "big number," "small number," "aligned number," "result of addition/subtraction," and "normalized number," respectively. The code is developed according to these stages, as shown in Listing 3.19.

Listing 3.19 Simplified floating-point adder

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity fp_adder is
5   port (
        sign1, sign2: in  std_logic;
        exp1, exp2: in  std_logic_vector(3 downto 0);
        frac1, frac2: in  std_logic_vector(7 downto 0);
        sign_out: out std_logic;
10        exp_out: out std_logic_vector(3 downto 0);
        frac_out: out std_logic_vector(7 downto 0)
    );
end fp_adder ;

15 architecture arch of fp_adder is
    -- suffix b, s, a, n for
    --    big, small, aligned, normalized number
    signal signb, signs: std_logic;
    signal expb, exps, expn: unsigned(3 downto 0);
20    signal fracb, fracs, fracn: unsigned(7 downto 0);
    signal sum_norm: unsigned(7 downto 0);
    signal exp_diff: unsigned(3 downto 0);
    signal sum: unsigned(8 downto 0); --one extra for carry
    signal lead0: unsigned(2 downto 0);
25 begin
    -- 1st stage: sort to find the larger number
    process (sign1, sign2, exp1, exp2, frac1, frac2)
    begin
        if (exp1 & frac1) > (exp2 & frac2) then
30            signb <= sign1;
            signs <= sign2;
            expb <= unsigned(exp1);
            exps <= unsigned(exp2);

```

```

        fracb <= unsigned(frac1);
35      fracs <= unsigned(frac2);
    else
        signb <= sign2;
        signs <= sign1;
        expb <= unsigned(exp2);
40      exps <= unsigned(exp1);
        fracb <= unsigned(frac2);
        fracs <= unsigned(frac1);
    end if;
end process;

45  -- 2nd stage: align smaller number
exp_diff <= expb - exps;
with exp_diff select
    fracb <=
50      fracs
        "0"          & fracs(7 downto 1) when "0000",
        "00"         & fracs(7 downto 2) when "0010",
        "000"        & fracs(7 downto 3) when "0011",
        "0000"       & fracs(7 downto 4) when "0100",
55      "00000"      & fracs(7 downto 5) when "0101",
        "000000"     & fracs(7 downto 6) when "0110",
        "0000000"    & fracs(7)      when "0111",
        "00000000"   when others;

60  -- 3rd stage: add/subtract
sum <= ('0' & fracb) + ('0' & fracb) when signb=signs else
        ('0' & fracb) - ('0' & fracb);

-- 4th stage: normalize
65  -- count leading 0s
lead0 <= "000" when (sum(7)='1') else
        "001" when (sum(6)='1') else
        "010" when (sum(5)='1') else
        "011" when (sum(4)='1') else
70      "100" when (sum(3)='1') else
        "101" when (sum(2)='1') else
        "110" when (sum(1)='1') else
        "111";

-- shift significant according to leading 0
75  with lead0 select
    sum_norm <=
        sum(7 downto 0) when "000",
        sum(6 downto 0) & '0' when "001",
        sum(5 downto 0) & "00" when "010",
80      sum(4 downto 0) & "000" when "011",
        sum(3 downto 0) & "0000" when "100",
        sum(2 downto 0) & "00000" when "101",
        sum(1 downto 0) & "000000" when "110",
        sum(0) & "0000000" when others;

85  -- normalize with special conditions

```

```

process (sum, sum_norm, expb, lead0)
begin
    if sum(8)='1' then -- w/ carry out; shift frac to right
90      expn <= expb + 1;
        fracn <= sum(8 downto 1);
    elsif (lead0 > expb) then -- too small to normalize;
        expn <= (others=>'0'); -- set to 0
        fracn <= (others=>'0');
95      else
        expn <= expb - lead0;
        fracn <= sum_norm;
    end if;
end process;
100
-- form output
sign_out <= signb;
exp_out <= std_logic_vector(expn);
frac_out <= std_logic_vector(fracn);
105 end arch;

```

The circuit in the first stage compares the magnitudes and routes the big number to the `signb`, `expb`, and `fracb` signals and the smaller number to the `signs`, `exps`, and `fracs` signals. The comparison is done between `exp1&frac1` and `exp2&frac2`. It implies that the exponents are compared first, and if they are the same, the significands are compared.

The circuit in the second stage performs alignment. It first calculates the difference between the two exponents, which is `expb-exps`, and then shifts the significand, `fracs`, to the right by this amount. The aligned significand is labeled `fracn`. The circuit in the third stage performs sign-magnitude addition, similar to that in Section 3.7.2. Note that the operands are extended by 1 bit to accommodate the carry-out bit.

The circuit in the fourth stage performs normalization, which adjusts the result to make the final output conform to the normalized format. The normalization circuit is constructed in three segments. The first segment counts the number of leading zeros. It is somewhat like a priority encoder. The second segment shifts the significands to the left by the amount specified by the leading-zero counting circuit. The last segment checks the carry-out and zero conditions and generates the final normalized number.

Testing circuit The floating-point adder has two 13-bit input operands. Since the prototyping board has only one 8-bit switch and four 1-bit pushbuttons, it cannot provide enough number of physical inputs to test the circuit. To accommodate the 26 bits of the floating-point adder, we must create a testing circuit and assign constants or duplicated switch signals to the adder's input operands. An example is shown in Listing 3.20. It assigns one operand as constant and uses duplicated switch signals for the other operand. The addition result is passed to the hexadecimal decoders and the sign circuit and is shown on the seven-segment LED display.

Listing 3.20 Floating-point adder testing circuit

```

library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
entity fp_adder_test is
5   port (

```

```

        clk: in std_logic;
        sw: in std_logic_vector(7 downto 0);
        btn: in std_logic_vector(3 downto 0);
        an: out std_logic_vector(3 downto 0);
10    sseg: out std_logic_vector(7 downto 0)
    );
end fp_adder_test;

architecture arch of fp_adder_test is
15    signal sign1, sign2: std_logic;
    signal exp1, exp2: std_logic_vector(3 downto 0);
    signal frac1, frac2: std_logic_vector(7 downto 0);
    signal sign_out: std_logic;
    signal exp_out: std_logic_vector(3 downto 0);
20    signal frac_out: std_logic_vector(7 downto 0);
    signal led3, led2, led1, led0:
        std_logic_vector(7 downto 0);
begin
    -- set up the fp adder input signals
25    sign1 <= '0';
    exp1 <= "1000";
    frac1 <= '1' & sw(1) & sw(0) & "10101";
    sign2 <= sw(7);
    exp2 <= btn;
30    frac2 <= '1' & sw(6 downto 0);

    -- instantiate fp adder
    fp_add_unit: entity work.fp_adder
        port map(
35            sign1=>sign1, sign2=>sign2, exp1=>exp1, exp2=>exp2,
            frac1=>frac1, frac2=>frac2,
            sign_out=>sign_out, exp_out=>exp_out,
            frac_out=>frac_out
        );
40

    -- instantiate three instances of hex decoders
    -- exponent
    sseg_unit_0: entity work.hex_to_sseg
        port map(hex=>exp_out, dp=>'0', sseg=>led0);
45    -- 4 LSBs of fraction
    sseg_unit_1: entity work.hex_to_sseg
        port map(hex=>frac_out(3 downto 0),
            dp=>'1', sseg=>led1);
    -- 4 MSBs of fraction
50    sseg_unit_2: entity work.hex_to_sseg
        port map(hex=>frac_out(7 downto 4),
            dp=>'0', sseg=>led2);

    -- sign
    led3 <= "11111110" when sign_out='1' else -- middle bar
55    "11111111"; -- blank

    -- instantiate 7-seg LED display time-multiplexing module
    disp_unit: entity work.disp_mux

```

```

port map(
60   clk=>clk, reset=>'0',
      in0=>led0, in1=>led1, in2=>led2, in3=>led3,
      an=>an, sseg=>sseg
);
end arch;

```

3.8 BIBLIOGRAPHIC NOTES

The Designer's Guide to VHDL by P. J. Ashenden provides detailed coverage on the VHDL constructs discussed in this chapter, and the author's *RTL Hardware Design Using VHDL: Coding for Efficiency, Portability, and Scalability* discusses the coding and optimization schemes and gives additional design examples.

3.9 SUGGESTED EXPERIMENTS

3.9.1 Multi-function barrel shifter

Consider an 8-bit shifting circuit that can perform rotating right or rotating left. An additional 1-bit control signal, *lr*, specifies the desired direction.

1. Design the circuit using one rotate-right circuit, one rotate-left circuit, and one 2-to-1 multiplexer to select the desired result. Derive the code.
2. Derive a testbench and use simulation to verify operation of the code.
3. Synthesize the circuit, program the FPGA, and verify its operation.
4. This circuit can also be implemented by one rotate-right shifter with pre- and post-reversing circuits. The reversing circuit either passes the original input or reverses the input bitwise (for example, if an 8-bit input is $a_7a_6a_5a_4a_3a_2a_1a_0$, the reversed result becomes $a_0a_1a_2a_3a_4a_5a_6a_7$). Repeat steps 2 and 3.
5. Check the report files and compare the number of logic cells and propagation delays of the two designs.
6. Expand the code for a 16-bit circuit and synthesize the code. Repeat steps 1 to 5.
7. Expand the code for a 32-bit circuit and synthesize the code. Repeat steps 1 to 5.

3.9.2 Dual-priority encoder

A dual-priority encoder returns the codes of the highest or second-highest priority requests. The input is a 12-bit *req* signal and the outputs are *first* and *second*, which are the 4-bit binary codes of the highest and second-highest priority requests, respectively.

1. Design the circuit and derive the code.
2. Derive a testbench and use simulation to verify operation of the code.
3. Design a testing circuit that displays the two output codes on the seven-segment LED display of the prototyping board, and derive the code.
4. Synthesize the circuit, program the FPGA, and verify its operation.

3.9.3 BCD incrementor

The binary-coded-decimal (BCD) format uses 4 bits to represent 10 decimal digits. For example, 259_{10} is represented as "0010 0101 1001" in BCD format. A BCD incrementor